Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre

A Project Report
on
"Graphify"

[Code No.: COMP 206]

(For partial fulfillment of Year II / Semester I in Computer Science E)

Submitted by
Binary Kumar Chongbang (37963-24)

Submitted to
Sagar Acharya
Department of Computer Science and Engineering

February 25, 2026

Bona fide Certificate

This project work on

“Graphify”

is the bona fide work of

“

Binary Kumar Chongbang (37963-24)

”

who carried out the project for the fullfillment of the mini-project for I/II.

Acknowledgements

I would like to express my sincere gratitude to the Department of Computer Science and Engineering, Kathmandu University, for providing the resources and environment necessary for carrying out this mini-project. I am also thankful to my friends and colleagues who provided motivation and encouragement during the course of this work. This project, **Graphify: Dijkstra Algorithm Visualization Tool**, is the result of my individual effort as part of the mini-project requirements.

I would also like to express my gratitude towards our lecturer Er.Sagar Acharya for providing us with opportunity to implement our theoretical knowledge to build a real-world application.

Abstract

Graphify is a desktop-based visualization tool developed using C++ and Qt to demonstrate the working of Dijkstra's shortest path algorithm. The system provides a graphical representation of weighted graphs and dynamically visualizes the step-by-step execution of the algorithm. It highlights node visits, edge relaxations, distance updates, and the final shortest path tree. The tool is designed for educational purposes to help students better understand graph traversal and shortest path computation.

Keywords: *Shortest Path, Data Structure and Algorithm, Dijkstra's*

Contents

Acknowledgements ii

Abstract iii

List of Figures vi

List of Tables vii

Abbreviations viii

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Objectives . . . 1
- 1.3 Motivation and Significance . . . 1

2 Related Works 2

3 Design and Implementation 3
- 3.1 System Overview . . . 3
- 3.2 System Requirement Specifications . . . 4
  - 3.2.1 Software Requirements . . . 4
  - 3.2.2 Hardware Requirements . . . 4
- 3.3 Implementation Details . . . 5
  - 3.3.1 Implemented Dijkstra's Algorithm . . . 5
  - 3.3.2 Dijkstra Algorithm Logic . . . 5
  - 3.3.3 Time Complexity . . . 5
  - 3.3.4 Software Implementation . . . 6
- 3.4 Algorithms and Flowcharts . . . 7
  - 3.4.1 Algorithm . . . 7
  - 3.4.2 Flowchart . . . 8
  - 3.4.3 Use-case diagram . . . 9

4 Results and Discussion 10
- 4.1 Implemented Features . . . 10
- 4.2 Results and Performance Analysis . . . 11
- 4.3 Comparison with Objectives or Existing Systems . . . 11
- 4.4 Challenges and Limitations . . . 12
- 4.5 Discussion . . . 12

5 Conclusion and Future Works 13
- 5.1 Future Enhancements . . . 13

References 14

Appendix A **15**
A.1 Additional Figures and Screenshots 15

List of Figures

3.1 System flowchart 8
3.2 Use-case diagram 9

5.1 Main UI 15
5.2 Input Panel 16
5.3 Traversal Table 16
5.4 Initial state 17
5.5 Intermediate State 17
5.6 Completion State 17

List of Tables

3.1 Minimum Hardware Requirements for Graphify 4

Abbreviations

API Graphical User Interface.

DSA Data Structure and Algorithm.

UI User Interface.

Chapter 1

Introduction

1.1 Background

Graph algorithms play a fundamental role in computer science, particularly in networking, transportation systems, and optimization problems. Among them, Dijkstra's algorithm is one of the most widely used methods for computing the shortest path in weighted graphs with non-negative edge weights.

Understanding Dijkstra's algorithm through static examples can be difficult for beginners. Therefore, this project focuses on developing a graphical visualization tool that allows users to observe how distances are updated and how the shortest path is formed step-by-step.

1.2 Objectives

- To implement Dijkstra's shortest path algorithm in C++
- To visualize graph nodes and edges dynamically
- To display distance updates in tabular format
- To make user friendly and interactive

1.3 Motivation and Significance

Many students struggle to understand Data Structure and Algorithm (DSA) because theoretical explanations do not clearly show how data changes during execution. This project was chosen to address that issue by developing an interactive DSA Visualizer that demonstrates the step-by-step behavior for Dijksta's algorithm.

The visualizer makes it easier to learn, experiment, and connect theory with practice, and it is designed to be extendable so additional algorithms and structures can be added in the future. Overall, the project provides an effective and engaging learning tool for computer science students.

Chapter 2

Related Works

Several projects and tools have been developed to help learners understand data structures and algorithms through visualizations. Visualgo (2018) is a popular web-based platform that provides interactive animations for graph traversal algorithms. While it offers extensive algorithm coverage, it is limited to online use and lacks a fully interactive desktop interface.

Also academic works have been proposed with Python-based visualization tools. For example, Sahu et al. (2025) presented a Python tool for sorting and graph algorithms with animations, and Kulkarni et al. (2023) developed an algorithm visualizer focusing on educational enhancement. These tools demonstrate the educational value of interactive visualization but are often limited in scope, covering only selected algorithms or lacking rich Graphical User Interface (API) features for multiple data structures.

This project aims to address these gaps by developing a desktop-based Graph visualizer using c++ and Qt, offering interactive, step-by-step animations for graphs. By combining offline accessibility, a user-friendly interface, and algorithm visualizations, this project provides all in one package for DSA learners.

- " Algorithm Visualizer (2024)" on GitHub demonstrated interactive animations for stacks, queues, and sorting algorithms using Python and Tkinter.

- Pranjali. (2025)Algo Visualix A Python-Based Algorithm Visualizer for Educational Enhancement. IJRASET: Journal for Research in Applied Science and Engineering Technology demonstrates use of python for visualization of algorithm and data structures.

In comparison to web-based or Tkinter-based implementations, developing the "Graphify" with C++ and Qt allows for a fully interactive modular desktop application. This approach provides better control over animations, user interactions, and future extensions, making it a more flexible and effective learning tool.

Chapter 3

Design and Implementation

It includes the explanation of sequential procedure that was performed during the project work. It includes algorithms, flowcharts, System Diagrams and other analysis and design which gives general idea about the different approach in development procedures during the project work.

3.1 System Overview

The system is composed of several major components:

- User Interface (UI): Built using Qt designer, it provides interactive controls for selecting data structures and algorithms and displaying the visualizations.
- Data Structure Modules: Graph is implemented as a separate module that handles the operations and internal data management.
- Algorithm Modules: Dijkstra's algorithm is implemented to manipulate the selected data structure and generate step-by-step changes for visualization.
- Animation and Visualization Engine: Handles the rendering of data and animations, highlighting changes in data elements as operations are performed.

The components interact in a modular way. When a user selects a data structure and algorithm from the UI, the corresponding modules execute the operations and send updates to the visualization engine, which dynamically animates each step. This design ensures flexibility, allowing new data structures or algorithms to be added easily.

3.2 System Requirement Specifications

3.2.1 Software Requirements

- Programming language: C++
- Frameworks and libraries: Qt
- Compiler: MinGW
- Development tools and environments: Qt Creator, Git
- Operating system: Windows 10/11(64 bit)

Functional Requirements

- step-by-step Visualization: The system displays each operation step by step, showing changes in the data structure dynamically.
- Interactive Controls: Users can start or reset the visualization and adjust the speed of animations.
- select/Modify Source: Users can input source for the graph to observe algorithm behavior on different inputs.

Non-Functional Requirements

- Performance: The system should render animations smoothly without significant lag, even for moderate-sized data sets.
- Usability: The interface should be intuitive and easy to navigate, with clearly labeled controls.

3.2.2 Hardware Requirements

Component	**Specification**
Processor	Dual Core
DirectX	9
RAM	2 GB
Available Storage	1 GB

Table 3.1: Minimum Hardware Requirements for Graphify

3.3 Implementation Details

3.3.1 Implemented Dijkstra's Algorithm

- Dijkstra's Algorithm: It is a single source shortest path finding algorithm developed by Edsger W. Dijkstra in 1956—reputedly during a twenty-minute tea break—it solved the fundamental problem of finding the most efficient path between two points in a network of connections. Its brilliance lies in its "greedy" philosophy, where it systematically identifies the shortest known distance to reach every destination, ensuring that the final route discovered is the shortest.

3.3.2 Dijkstra Algorithm Logic

1. **Initialize:** Set the distance to all nodes to ∞.
2. **Source:** Set the distance of the starting (source) node to 0.
3. **Repeat:**
    - Select the unvisited node with the minimum distance.
    - Mark this node as visited.
    - Relax all adjacent edges: update the distance to each neighbor if the path through the current node is shorter.
4. **Terminate:** Continue the process until all nodes have been processed or the destination is reached.

3.3.3 Time Complexity

$$O((V + E) \log V)$$

Where:

- $V =$ Number of vertices
- $E =$ Number of edges

3.3.4 Software Implementation

Graphify is implemented as a desktop application using C++ and Qt, providing an interactive environment to visualize graph. The implementation follows a modular design, separating the user interface, data structure modules, algorithm modules, and the visualization engine.

Key Implementation Steps:

- Module Design: Each data structure was implemented as an independent module, handling its operations and internal data management.
- Algorithm Implementation: Dijkasta's algorithm was implemented to manipulate the selected data structures.
- The UI was created using Qt designer and Qt's mainwindow.ui, layouts, and graphics modules. Controls allow users to select source node and control the visualization
- Visualization Engine: Animations are rendered using QGraphicsView and QGraphicsScene, showing step-by-step updates of the data structures and highlighting changes during algorithm execution.

Technical Decisions:

- Qt was chosen over other for its support of graphics, animation, and cross-platform compatibility.
- Animations were implemented carefully to maintain performance and ensure clear visualization, even for moderately sized datasets.

3.4 Algorithms and Flowcharts

3.4.1 Algorithm

1. **Start the Program**
   (a) Initialize the Qt application.
   (b) Load the graphical user interface.
   (c) Display the predefined graph.
2. **User Input Handling**
   (a) Allow the user to select a source node
   (b) Accept input data from the user.
   (c) Validate the input.
   (d) Display an error message if the input is invalid.
3. **Algorithm Execution**
   (a) Execute the algorithm step-by-step.
   (b) Update the internal state after each step.
   (c) Send the updated state to the visualization module.
4. **Visualization Process**
   (a) Convert data structure states into visual elements.
   (b) Animate operations such as selection, comparison, visit and visited.
   (c) Update the GUI with appropriate time delays.
5. **Display Result**
   (a) Display the final result of the algorithm.
   (b) Show traversal table.
6. **Terminate Program**
   (a) Release allocated resources.
   (b) Close the application safely.

3.4.2 Flowchart

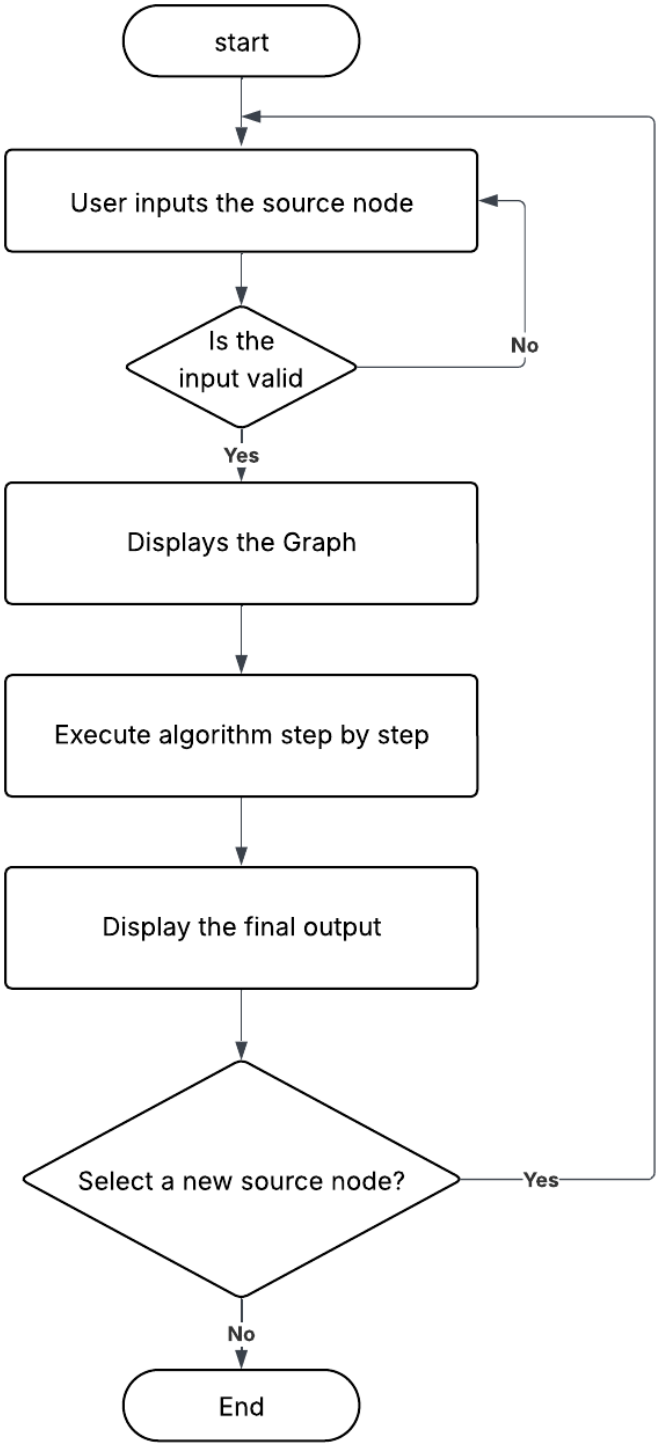

Figure 3.1: System flowchart

3.4.3 Use-case diagram

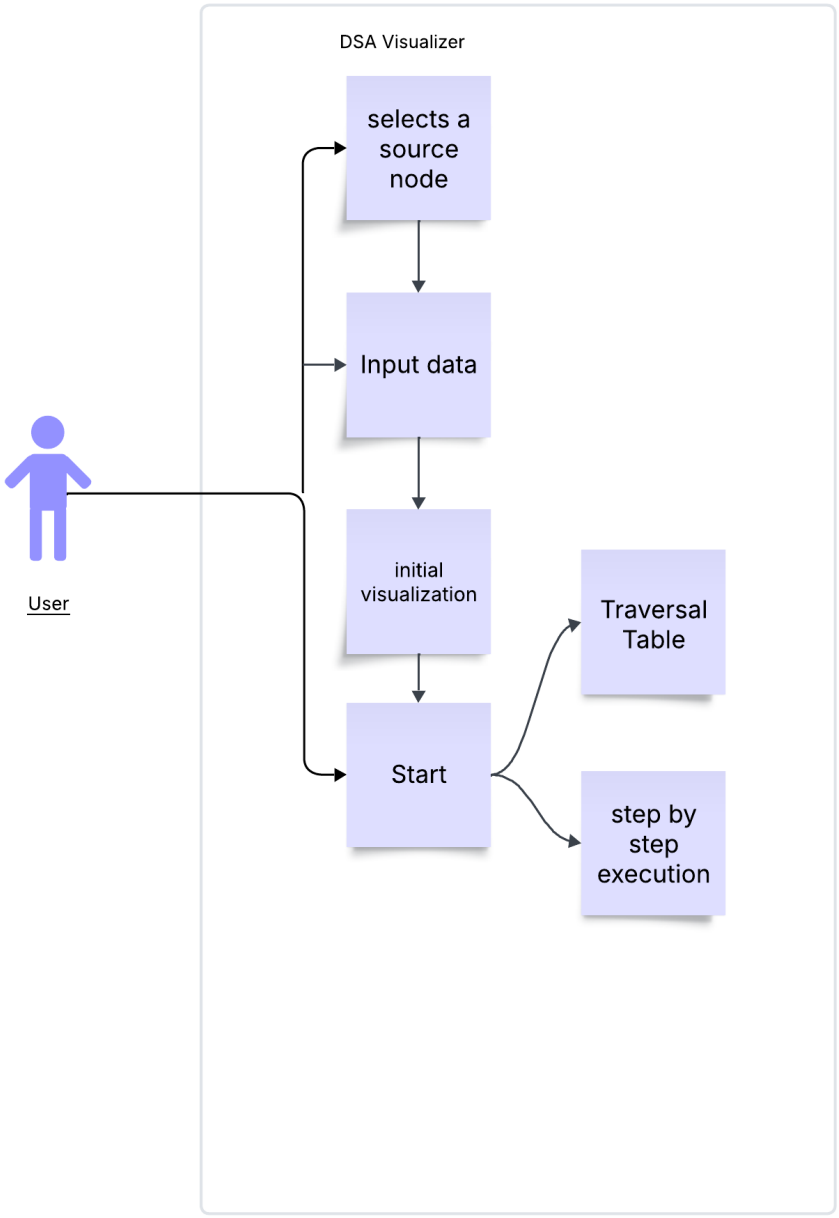

Figure 3.2: Use-case diagram

Chapter 4

Results and Discussion

This chapter presents the results obtained from the completion of the Graphify and discusses their significance in understanding data structures and algorithms. The visualizations demonstrate how different operations affect data structures step by step, allowing users to observe the internal behavior of algorithms in real time. The chapter highlights the key features of the system, evaluates its performance and usability, and compares the outcomes with the expected behavior of the algorithms.

4.1 Implemented Features

The DSA Visualizer includes several key features designed to help users understand shortest path algorithm interactively:

- **Algorithm Execution**:Dijkstra's shortest path algorithm can be executed step by step to demonstrate how data is manipulated internally.
- **Step-by-Step Visualization**: Operations are animated using Qt's graphics modules, highlighting changes in the data structure at each step. This makes it easier for users to follow and understand algorithm flow.
- **Interactive Controls**: Users can start, input, and clear visualization, providing flexibility and control over the learning process.
- **Custom Data Input**: Users can input their own datasets to test and observe algorithm behavior on different inputs, enhancing experimentation and learning.

4.2 Results and Performance Analysis

The implemented Visualizer successfully demonstrates the behavior of Dijkstra's algorithms through interactive, step-by-step visualizations. Key functionalities such inputting custom data, and controlling the animation work as expected.

The system performs efficiently for moderately sized datasets with smooth animations and minimal lag. Correctness of the visualizations was verified by comparing the outputs with expected algorithm behavior. The modular design ensures that adding new data structures or algorithms does not affect existing functionalities, demonstrating scalability and maintainability.

Overall, the results confirm that the"Graphify" effectively bridges the gap between theoretical understanding and practical observation, making it a useful learning tool for students.

4.3 Comparison with Objectives or Existing Systems

"Graphify" successfully fulfills its main objectives by allowing users to interact, input custom data, and observe step-by-step visualizations of algorithm operations. Its modular design also enables future expansion, making it possible to add new data structures and algorithms without affecting existing functionality. The GUI is user-friendly with controlled execution.

Compared to existing systems like VisuAlgo or Tkinter-based visualizers, this project provides a desktop-based, interactive platform with smooth real-time animations using Qt and C++, giving users better control over visualization and interaction. While some web-based systems support larger datasets,Graphify is optimized for clarity and educational value, handling moderate dataset sizes without performance issues. Since it is a desktop based application there is no need for internet connection. The visualization steps are also different and beginners friendly.

Overall, the system aligns well with its original objectives, improves upon existing solutions by emphasizing usability and interactivity, and serves as a flexible and practical tool for learning data structures and algorithms.

4.4 Challenges and Limitations

During the development of Graphify, some of the challenges faced are:

- **Ensuring smooth real-time animations**: Rendering multiple steps could cause performance delays for larger datasets, so the system was optimized for moderate sizes.
- **visualization of algorithms**: Animating the algorithm so that even a beginner could understand was a key task so, developing the way for the algorithm to be visualized was a key focal point.
- **Synchronization and Animation**: Controlling animation speed so that users can follow steps but still have smooth visualization.Synchronizing multiple visual changes (like node color change, movement, and text update) to avoid glitches.
- **Integration Challenges**: Connecting backend algorithm logic with frontend while ensuring no errors or delays was complicated. It was possible by maintaining a clean separation between data handling and UI rendering.

Some of the limitations of Graphify are:

- Does not include features such as node and edge creation by simple click and drag.
- Consists of only Dijkstra's algorithm for the time being.
- It does not support cross platform for other os such as linux or mac. It only works on windows for the time being.

4.5 Discussion

Graphify successfully demonstrates the behavior of Dijkstra's algorithms through interactive, step-by-step visualizations. The outcomes align with the primary project objectives of helping users understand how data changes during algorithm execution.Some limitations were observed during testing. The system's Performance also decreases with larger datasets, limiting the number of elements that can be visualized smoothly.

Despite these constraints, the project provides meaningful insights into algorithm behavior, making it an effective educational tool and providing a foundation for future improvements.

Chapter 5

Conclusion and Future Works

With the completion of this project and meeting it's objective of visualization of the Dijkstra's algorithm, it was possible to bridge the gap between the theory and practical application of DSA. During development, challenges such as algorithm optimization, real-time visualization, and user interaction were addressed, resulting in a tool that is both educational and user-friendly. Overall, this project not only reinforces the understanding of fundamental and advanced data structures but also provides a practical platform for learning and experimentation, making it a valuable resource for students and programming enthusiasts.

During the process of developing I was able to gain insights on new topics, explore the new paths and gain new experience that has improved me in more than one way. There are also some few shortcomings that shall be improved in future with addition of new features.

5.1 Future Enhancements

1. **Support for Larger Datasets**: Optimize animations and data handling to handle bigger datasets without performance issues.

2. **Advanced Data Structures**: Include more complex structures like balanced trees, weighted graphs, dynamic graphs and other algorithms.

3. **Enhanced User Interface**: Add features such as algorithm comparison, Time and Space complexities calculations, detailed explanations, and interactive tutorials.

4. **Cross-Platform Deployment**: Package the application for multiple operating systems such as linxu and MacOS and consider a web-based version for wider accessibility.

References

Algorithm Visualizer (2024). Algorithm visualizer. `https://github.com/algorithm-visualizer/algorithm-visualizer`. GitHub repository.

Pranjali., S. . (2025). Algo visualix: A python-based algorithm visualizer for educational enhancement. *Algo Visualix: A Python-Based Algorithm Visualizer for Educational Enhancement. IJRASET: Journal for Research in Applied Science and Engineering Technology.*

Visualgo (2018). Visualgo. *Visualizing Data Structures and Algorithms through Animation. https://visualgo.net*.

Appendix A

The appendix contains supplementary material that supports the main content of the report:

A.1 Additional Figures and Screenshots

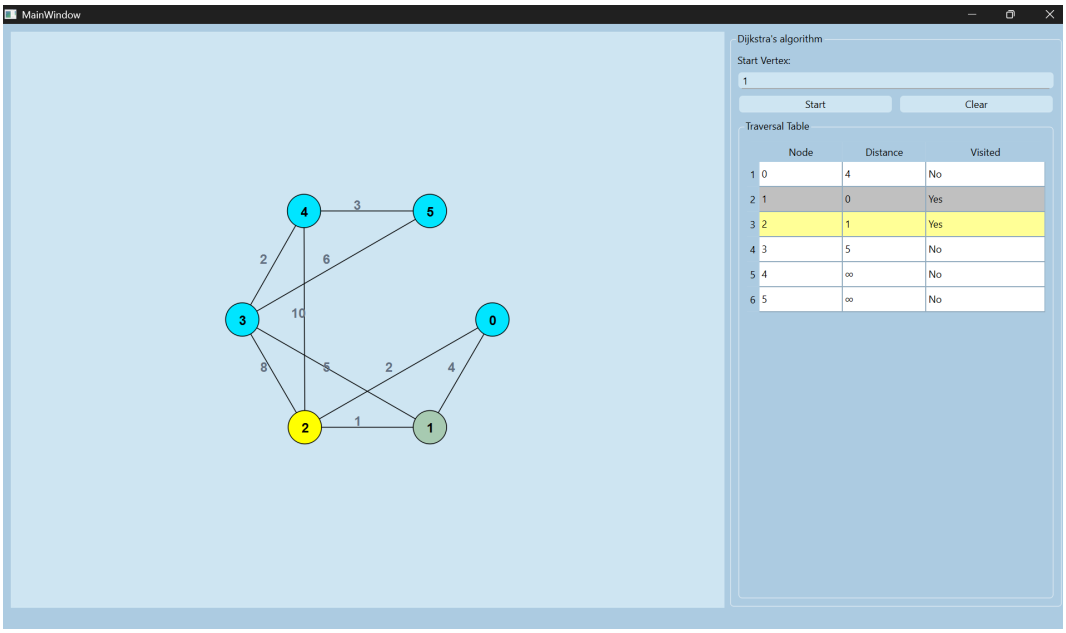

Figure 5.1: Main UI

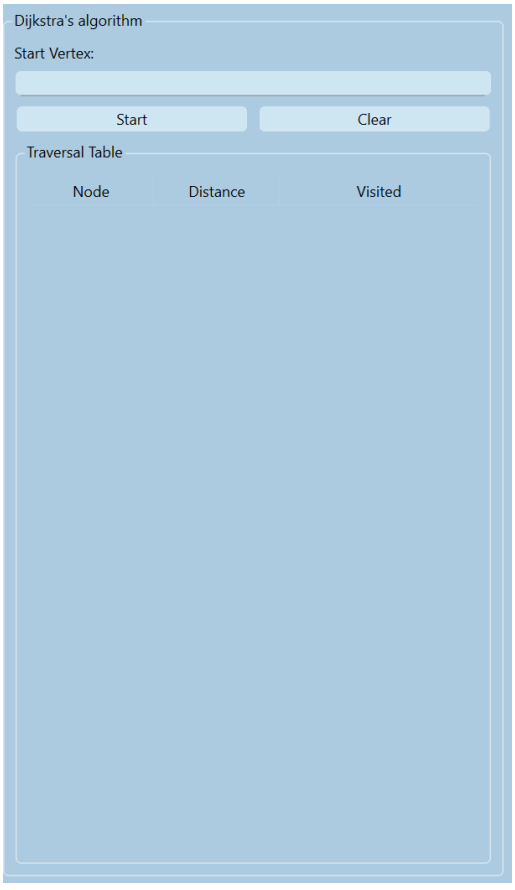

Figure 5.2: Input Panel

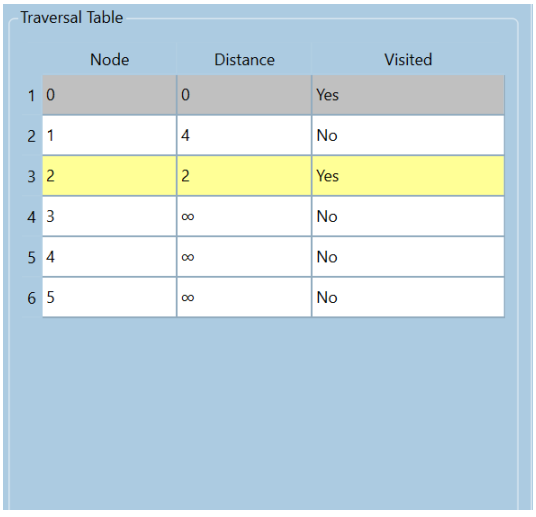

Figure 5.3: Traversal Table

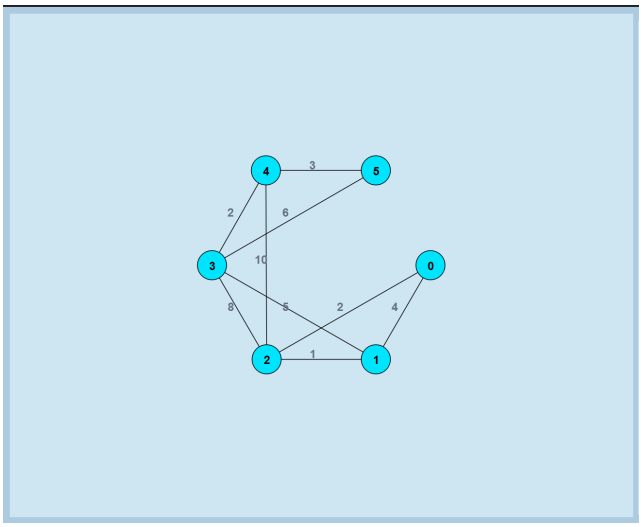

Figure 5.4: Initial state

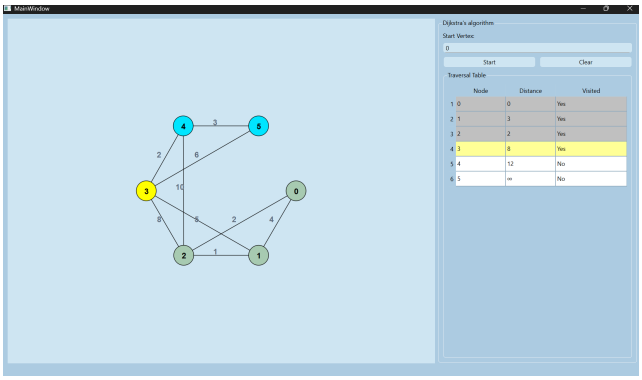

Figure 5.5: Intermediate State

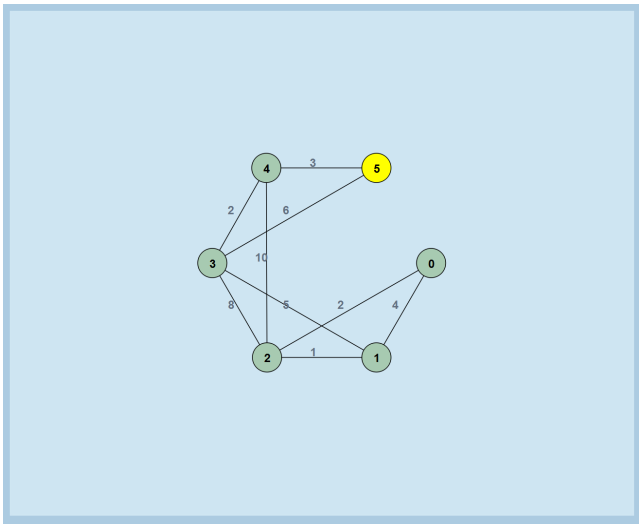

Figure 5.6: Completion State